

=====

BIBLE OPENPYXL – RÉFÉRENCE EXHAUSTIVE DES FONCTIONS

LÉGENDE : [wb] = Workbook | [ws] = Worksheet | [cell] = Cell | [op] = openpyxl

=====

[op] Classeur – Workbook

DESCRIPTION : Crée, charge et sauvegarde des fichiers Excel (.xlsx)
POURQUOI : Point d'entrée de tout fichier Excel avec openpyxl
CONTEXTE : Création de rapports, édition de fichiers existants, automation

CRÉATION / CHARGEMENT :

```
from openpyxl import Workbook, load_workbook
wb = Workbook()                               Nouveau classeur (1 feuille "Sheet")
wb = Workbook(write_only=True)                 Mode écriture seule (gros fichiers)
wb = Workbook(iso_dates=True)                 Dates au format ISO 8601
wb = load_workbook("fichier.xlsx")            Charge un fichier existant
wb = load_workbook("fichier.xlsx", data_only=True) Valeurs calculées (pas formules)
wb = load_workbook("fichier.xlsx", read_only=True) Lecture seule (économise RAM)
wb = load_workbook("fichier.xlsx", keep_vba=True) Conserve les macros VBA
wb = load_workbook("fichier.xlsx", keep_links=True) Conserve les liens externes
wb = load_workbook("fichier.xlsx", rich_text=True) Conserve le rich text
wb = load_workbook(io.BytesIO(bytes_data))    Depuis un objet bytes en mémoire
```

SAUVEGARDE :

```
wb.save("sortie.xlsx")                        Sauvegarde dans un fichier
wb.save(io.BytesIO())                        Sauvegarde en mémoire
wb.close()                                    Ferme le classeur (libère ressources)
```

PROPRIÉTÉS DU CLASSEUR :

```
wb.active                                    Feuille active courante
wb.active = wb["Feuille2"]                  Change la feuille active
wb.sheetnames                               Liste des noms de feuilles
wb.worksheets                               Liste des objets Worksheet
wb.read_only                                True si ouvert en lecture seule
wb.write_only                               True si ouvert en écriture seule
wb.data_only                                True si valeurs seules (pas formules)
wb.template                                 True si le fichier est un template .xltx
wb.encoding                                 Encodage du fichier
```

MÉTADONNÉES (PROPERTIES) :

```
wb.properties.title                         Titre du document
wb.properties.subject                       Sujet
wb.properties.creator                       Auteur
wb.properties.description                   Description
wb.properties.keywords                      Mots-clés
wb.properties.category                     Catégorie
wb.properties.lastModifiedBy                Dernier modificateur
wb.properties.created                       Date de création (datetime)
wb.properties.modified                     Date de modification (datetime)
wb.properties.contentStatus                 Statut du contenu
wb.properties.revision                      Révision
wb.properties.version                      Version
wb.properties.language                     Langue
```

SÉCURITÉ :

```
wb.security.workbookPassword               Mot de passe classeur
wb.security.lockStructure                   Verrouille la structure
wb.security.lockWindows                     Verrouille les fenêtres
wb.security.lockRevision                    Verrouille les révisions
```

CALCUL :

```
wb.calculation.calcMode                    "auto", "manual", "autoNoTable"
wb.calculation.fullCalcOnLoad              True = recalcul à l'ouverture
wb.calculation.iterate                     True = calcul itératif (circulaires)
wb.calculation.iterateCount                Nb max d'itérations (défaut 100)
wb.calculation.iterateDelta                Delta max (défaut 0.001)
```

THÈME / COULEURS NOMMÉES :

```
wb.loaded_theme                            Données XML du thème chargé
```

EXEMPLES :

```
wb = Workbook()
ws = wb.active
```

```

ws.title = "Données"
ws["A1"] = "Bonjour"
wb.save("rapport.xlsx")
wb.close()

# Chargement et modification
wb = load_workbook("existant.xlsx")
ws = wb["Feuille1"]
ws["B2"] = "=SUM(B2:B10)"
wb.save("modifié.xlsx")

# Depuis/vers mémoire
import io
buffer = io.BytesIO()
wb.save(buffer)
buffer.seek(0)
wb2 = load_workbook(buffer)

```

[ws] Feuilles – Worksheet

DESCRIPTION : Gère les feuilles de calcul et leur structure
POURQUOI : Organiser les données en onglets, manipuler la structure
CONTEXTE : Multi-feuilles, copie, navigation, configuration

GESTION DES FEUILLES :

<code>wb.create_sheet("Nom")</code>	Ajoute une feuille en fin
<code>wb.create_sheet("Nom", 0)</code>	Insère en position 0 (début)
<code>wb.create_sheet("Nom", -1)</code>	Insère avant la dernière
<code>wb.copy_worksheet(ws)</code>	Copie une feuille dans le même classeur
<code>wb.remove(ws)</code>	Supprime une feuille
<code>wb.remove(wb["Nom"])</code>	Supprime par nom
<code>wb.move_sheet("Nom", offset=1)</code>	Déplace de n positions
<code>wb["NomFeuille"]</code>	Accès par nom
<code>wb.worksheets[0]</code>	Accès par index
<code>for ws in wb.worksheets</code>	Itération sur toutes les feuilles

PROPRIÉTÉS DE LA FEUILLE :

<code>ws.title</code>	Nom de la feuille
<code>ws.sheet_state</code>	"visible", "hidden", "veryHidden"
<code>ws.sheet_state = "hidden"</code>	Masque la feuille
<code>ws.tab_color</code>	Couleur de l'onglet (ex: "FF0000")
<code>ws.tab_color = "FF0000"</code>	Onglet rouge
<code>ws.tab_color = openpyxl.styles.Color("FF0000")</code>	Via objet Color
<code>ws.sheet_view.showGridLines</code>	True/False (afficher grille)
<code>ws.sheet_view.showRowColHeaders</code>	True/False (en-têtes lignes/colonnes)
<code>ws.sheet_view.zoomScale</code>	Zoom en % (ex: 75, 100, 150)
<code>ws.sheet_view.tabSelected</code>	True si onglet sélectionné
<code>ws.sheet_view.view</code>	"normal", "pageBreakPreview", "pageLayout"
<code>ws.sheet_view.rightToLeft</code>	True = affichage RTL
<code>ws.sheet_view.topLeftCell</code>	Cellule en haut à gauche visible
<code>ws.sheet_view.showZeros</code>	True = afficher les zéros

DIMENSIONS :

<code>ws.dimensions</code>	Chaîne ex: "A1:Z100"
<code>ws.min_row</code>	Première ligne avec données
<code>ws.max_row</code>	Dernière ligne avec données
<code>ws.min_column</code>	Première colonne avec données
<code>ws.max_column</code>	Dernière colonne avec données
<code>ws.calculate_dimension()</code>	Recalcule les dimensions

PROPRIÉTÉS D'IMPRESSION :

<code>ws.page_setup.orientation</code>	"portrait" ou "landscape"
<code>ws.page_setup.paperSize</code>	1=Letter, 9=A4, 13=A3...
<code>ws.page_setup.fitToPage</code>	True = ajuster à la page
<code>ws.page_setup.fitToWidth</code>	Nb pages en largeur (0=auto)
<code>ws.page_setup.fitToHeight</code>	Nb pages en hauteur (0=auto)
<code>ws.page_setup.scale</code>	Mise à l'échelle en %
<code>ws.page_setup.firstPageNumber</code>	Numéro de la première page
<code>ws.page_margins.left</code>	Marge gauche en pouces
<code>ws.page_margins.right</code>	Marge droite en pouces
<code>ws.page_margins.top</code>	Marge haute en pouces
<code>ws.page_margins.bottom</code>	Marge basse en pouces
<code>ws.page_margins.header</code>	Marge en-tête

ws.page_margins.footer	Marge pied de page
ws.print_title_rows	Ex: "1:2" (répète les 2 premières lignes)
ws.print_title_cols	Ex: "A:B" (répète colonnes A et B)
ws.print_area	Ex: "A1:H50" (zone d'impression)
ws.set_printer_settings(paper_size, orientation)	Raccourci
ws.oddHeader.center.text	Texte centré en-tête (pages impaires)
ws.oddFooter.right.text	Texte droit pied de page
ws.evenHeader.left.text	En-tête gauche pages paires
ws.firstHeader.center.text	En-tête de la première page
# Codes spéciaux dans en-têtes/pieds :	
# &P = numéro de page, &N = nb total pages, &D = date, &T = heure	
# &F = nom fichier, &A = nom feuille, &B = gras, &I = italique, &U = souligné	
 FIGER LES VOLETS :	
ws.freeze_panes = "B2"	Figé la ligne 1 et la colonne A
ws.freeze_panes = "A2"	Figé la ligne 1 seulement
ws.freeze_panes = "B1"	Figé la colonne A seulement
ws.freeze_panes = None	Désactive le gel
 ZOOM ET SÉLECTION :	
ws.sheet_view.zoomScale = 80	Zoom à 80%
ws.sheet_view.zoomScaleNormal = 100	Zoom normal
ws.sheet_view.zoomScalePageBreakView = 75	Zoom aperçu sauts de page
ws.sheet_view.selection[0].activeCell = "B2"	Cellule active
ws.sheet_view.selection[0].sqref = "B2:D5"	Sélection multiple
 SAUTS DE PAGE :	
ws.row_breaks.append(Break(id=10))	Saut de page après ligne 10
ws.col_breaks.append(Break(id=3))	Saut de page après colonne 3
 PROTECTION DE FEUILLE :	
ws.protection.sheet	True = feuille protégée
ws.protection.password = "motdepasse"	Mot de passe (hash auto)
ws.protection.set_password("motdepasse")	Définit et hash le mdp
ws.protection.enable()	Active la protection
ws.protection.disable()	Désactive la protection
ws.protection.selectLockedCells	False = interdit sélection cellules verrouillées
ws.protection.selectUnlockedCells	False = interdit sélection cellules déverrouillées
ws.protection.formatCells	True = autorise formater
ws.protection.insertRows	True = autorise insérer lignes
ws.protection.deleteRows	True = autorise supprimer lignes
ws.protection.sort	True = autorise trier
ws.protection.autoFilter	True = autorise filtres automatiques
 EXEMPLES :	
ws = wb.create_sheet("Résumé", 0)	
ws.title = "Résumé"	
ws.tab_color = "1F4E79"	
ws.freeze_panes = "B2"	
ws.page_setup.orientation = "landscape"	
ws.page_setup.fitToPage = True	
ws.page_setup.fitToWidth = 1	
ws.print_title_rows = "1:1"	
ws.oddFooter.center.text = "Page &P sur &N"	

[cell] Cellules – Cell / accès & écriture

DESCRIPTION : Lit et écrit les données dans les cellules
POURQUOI : Cœur de la manipulation de données dans Excel
CONTEXTE : Saisie de données, formules, types de valeurs

ACCÈS AUX CELLULES :

ws["A1"]	Cellule A1
ws["A1"].value	Valeur de A1
ws["A1"] = "Bonjour"	Écriture directe
ws.cell(row=1, column=1)	Accès par index (1-based)
ws.cell(row=1, column=1, value="Texte")	Accès + écriture
ws.cell(row=2, column=3).value	Lecture par index
ws["A1:C3"]	Plage de cellules (tuple de tuples)
ws["A"]	Toute la colonne A
ws[1]	Toute la ligne 1
ws["A1":"C3"]	Plage (via slice)

ITÉRATION :

for row in ws.iter_rows():	Toutes les lignes
for row in ws.iter_rows(min_row=2):	Depuis ligne 2
for row in ws.iter_rows(min_row=1, max_row=10, min_col=1, max_col=5):	
for row in ws.iter_rows(values_only=True):	Valeurs seulement (pas objets Cell)
for col in ws.iter_cols():	Toutes les colonnes
for col in ws.iter_cols(min_col=2, max_col=4, values_only=True):	
for row in ws.rows:	Générateur de lignes
for col in ws.columns:	Générateur de colonnes
for row in ws:	Toutes les lignes (alias ws.rows)

ÉCRITURE EN MASSE :

ws.append([1, 2, 3])	Ajoute une ligne en fin
ws.append(["Nom", "Age", "Ville"])	Ligne d'en-tête
ws.append({"A": "Val1", "C": "Val2"})	Par lettre de colonne
ws.append({1: "Val1", 3: "Val3"})	Par index de colonne

TYPES DE VALEURS :

ws["A1"] = "Texte"	Chaîne de caractères
ws["A1"] = 42	Entier
ws["A1"] = 3.14	Flottant
ws["A1"] = True	Booléen
ws["A1"] = None	Cellule vide
ws["A1"] = datetime(2024, 1, 15)	Date/heure Python
ws["A1"] = date(2024, 1, 15)	Date Python
ws["A1"] = time(14, 30, 0)	Heure Python
ws["A1"] = timedelta(days=5)	Durée
ws["A1"] = "=SUM(B1:B10)"	Formule

PROPRIÉTÉS CELL :

cell.value	Valeur de la cellule
cell.row	Numéro de ligne (1-based)
cell.column	Numéro de colonne (1-based)
cell.column_letter	Lettre de colonne ("A", "B"...)
cell.coordinate	Coordonnée ex: "B3"
cell.data_type	"n"=number, "s"=string, "b"=bool...
cell.is_date	True si la cellule est une date
cell.number_format	Format numérique ex: "0.00"
cell.font	Objet Font
cell.fill	Objet PatternFill/GradientFill
cell.border	Objet Border
cell.alignment	Objet Alignment
cell.protection	Objet Protection
cell.comment	Objet Comment ou None
cell.hyperlink	Lien hypertexte ou None
cell.internal_value	Valeur interne brute
cell.encoded_value	Valeur encodée
cell.has_style	True si style non par défaut
cell.parent	Feuille parente
cell.offset(row=1, column=1)	Cellule décalée de (row, col)

CELLULES FUSIONNÉES :

ws.merge_cells("A1:C3")	Fusionne A1 à C3
ws.merge_cells(start_row=1, start_column=1, end_row=3, end_column=3)	
ws.unmerge_cells("A1:C3")	Défusionne
ws.unmerge_cells(start_row=1, start_column=1, end_row=3, end_column=3)	
ws.merged_cells	Objet contenant les plages fusionnées
ws.merged_cells.ranges	Liste des plages fusionnées

COMMENTAIRES :

from openpyxl.comments import Comment	
commentaire = Comment("Mon commentaire", "Auteur")	
commentaire.width = 200	Largeur du commentaire (px)
commentaire.height = 100	Hauteur du commentaire (px)
ws["A1"].comment = commentaire	Attache le commentaire
ws["A1"].comment = None	Supprime le commentaire
ws["A1"].comment.text	Texte du commentaire
ws["A1"].comment.author	Auteur du commentaire

HYPERLIENS :

ws["A1"].hyperlink = "https://example.com"	Lien URL
ws["A1"].hyperlink = "Feuille2!A1"	Lien interne
ws["A1"].hyperlink.target	URL du lien
ws["A1"].hyperlink.tooltip	Infobulle

CONVERSION DE COLONNES :

```
from openpyxl.utils import get_column_letter, column_index_from_string
get_column_letter(1)           "A"
get_column_letter(26)          "Z"
get_column_letter(27)          "AA"
column_index_from_string("A")   1
column_index_from_string("Z")   26
column_index_from_string("AA")  27
from openpyxl.utils import coordinate_to_tuple
coordinate_to_tuple("B3")       (3, 2) # (row, col)
from openpyxl.utils import absolute_coordinate
absolute_coordinate("A1")        "$A$1"
from openpyxl.utils.cell import range_boundaries
range_boundaries("A1:C3")        (1, 1, 3, 3) # (min_col,min_row,max_col,max_row)
```

EXEMPLES :

```
ws["A1"] = "Produit"
ws["B1"] = "Prix"
ws["C1"] = "Quantité"
ws["D1"] = "Total"
for i, (produit, prix, qte) in enumerate(données, start=2):
    ws.cell(row=i, column=1, value=produit)
    ws.cell(row=i, column=2, value=prix)
    ws.cell(row=i, column=3, value=qte)
    ws.cell(row=i, column=4, value=f"B{i}*C{i}")
```

```
# Lire toutes les données
données = list(ws.iter_rows(min_row=2, values_only=True))
```

```
# Fusion + centrage
ws.merge_cells("A1:D1")
ws["A1"] = "Titre du tableau"
ws["A1"].alignment = Alignment(horizontal="center")
```

[cell] Styles – Font, Fill, Border, Alignment, NumberFormat

DESCRIPTION : Applique des styles visuels aux cellules
POURQUOI : Mise en forme professionnelle, lisibilité, conformité
CONTEXTE : Rapports, tableaux, en-têtes, indicateurs visuels

— POLICE (Font) —

```
from openpyxl.styles import Font

Font(name="Arial")           Nom de la police
Font(name="Calibri", size=11) Taille en points
Font(bold=True)              Gras
Font(italic=True)            Italique
Font(underline="single")     Souligné simple
Font(underline="double")     Souligné double
Font(underline="singleAccounting") Souligné comptable simple
Font(underline="doubleAccounting") Souligné comptable double
Font(strike=True)            Barré
Font(color="FF0000")          Couleur rouge (hex ARGB)
Font(color="00FF0000")        Avec canal alpha (ARGB)
Font(vertAlign="superscript") Exposant
Font(vertAlign="subscript")   Indice
Font(vertAlign="baseline")    Normale
Font(charset=0)               Jeu de caractères
Font(underline=True)          Contour (Mac uniquement)
Font(shadow=True)             Ombre (Mac uniquement)
Font(condense=True)           Condensé
Font(extend=True)             Étendu
Font(scheme="minor")          Schéma de police (minor/major)
```

```
# Application
cell.font = Font(bold=True, color="1F4E79", size=14)
```

— REMPLISSAGE (Fill) —

```
from openpyxl.styles import PatternFill, GradientFill

PatternFill(fill_type="solid", start_color="FFFF00")  Jaune uni
PatternFill(fill_type="solid", fg_color="FFFF00")     Alias fgColor
PatternFill(fill_type=None)                            Pas de remplissage
```

PatternFill(fill_type="darkGray")	Gris foncé
PatternFill(fill_type="mediumGray")	Gris moyen
PatternFill(fill_type="lightGray")	Gris clair
PatternFill(fill_type="gray125")	Gris 12.5%
PatternFill(fill_type="gray625")	Gris 6.25%
PatternFill(fill_type="darkHorizontal")	Lignes horizontales
PatternFill(fill_type="darkVertical")	Lignes verticales
PatternFill(fill_type="darkDown")	Hachures descendantes
PatternFill(fill_type="darkUp")	Hachures montantes
PatternFill(fill_type="darkGrid")	Grille sombre
PatternFill(fill_type="darkTrellis")	Treillis sombre
PatternFill(fill_type="lightHorizontal")	Lignes horizontales claires
PatternFill(fill_type="lightVertical")	Lignes verticales claires
PatternFill(fill_type="lightDown")	Hachures claires descend.
PatternFill(fill_type="lightUp")	Hachures claires montantes
PatternFill(fill_type="lightGrid")	Grille claire
PatternFill(fill_type="lightTrellis")	Treillis clair
PatternFill(fill_type="solid", start_color="4472C4", end_color="FFFFFF")	Couleur de fond + motif

GradientFill(type="linear", degree=0,	Dégradé linéaire
stop=[GradientStop(position=0, color="FFFFFF"),	
GradientStop(position=1, color="4472C4")])	
GradientFill(type="path",	Dégradé radial
stop=[GradientStop(position=0, color="FFFFFF"),	
GradientStop(position=1, color="4472C4")])	

```
# Application
cell.fill = PatternFill("solid", fgColor="4472C4")
```

— BORDURES (Border) —
from openpyxl.styles import Border, Side

# Styles de trait	
Side(style=None)	Pas de bordure
Side(style="thin")	Fin
Side(style="medium")	Moyen
Side(style="thick")	Épais
Side(style="double")	Double
Side(style="hair")	Très fin (cheveu)
Side(style="dotted")	Pointillé
Side(style="dashed")	Tirets
Side(style="dashDot")	Tiret-point
Side(style="dashDotDot")	Tiret-point-point
Side(style="slantDashDot")	Tiret oblique
Side(style="mediumDashed")	Tirets moyen
Side(style="mediumDashDot")	Tiret-point moyen
Side(style="mediumDashDotDot")	Tiret-point-point moyen
Side(style="thin", color="FF0000")	Fin rouge

Border(left=Side(style="thin"), right=Side(style="thin"), top=Side(style="thin"), bottom=Side(style="thin"))	Bordure complète fine
---	-----------------------

Border(outline=True)	Contour seulement
Border(left=Side(style="medium"), right=Side(style="medium"), top=Side(style="thick"), bottom=Side(style="thick"))	Bordures mixtes

# Diagonales	
Border(diagonal=Side(style="thin"), diagonalUp=True)	Diagonale montante
Border(diagonal=Side(style="thin"), diagonalDown=True)	Diagonale descendante

```
# Application
fin = Side(style="thin")
cell.border = Border(left=fin, right=fin, top=fin, bottom=fin)
```

```
# Appliquer une bordure à une plage
def border_range(ws, range_string):
    from openpyxl.styles import Border, Side
    fine = Side(style="thin")
```

```

epaisse = Side(style="medium")
rows = list(ws[range_string])
for i, row in enumerate(rows):
    for j, cell in enumerate(row):
        top    = epaisse if i == 0 else fine
        bottom = epaisse if i == len(rows)-1 else fine
        left   = epaisse if j == 0 else fine
        right  = epaisse if j == len(row)-1 else fine
        cell.border = Border(top=top, bottom=bottom, left=left, right=right)

```

— ALIGNEMENT (Alignment) —

```
from openpyxl.styles import Alignment
```

Alignment(horizontal="left")	Gauche
Alignment(horizontal="center")	Centré
Alignment(horizontal="right")	Droite
Alignment(horizontal="fill")	Remplissage
Alignment(horizontal="justify")	Justifié
Alignment(horizontal="centerContinuous")	Centre continu
Alignment(horizontal="distributed")	Distribué
Alignment(horizontal="general")	Général (par défaut Excel)
Alignment(vertical="top")	Haut
Alignment(vertical="center")	Milieu
Alignment(vertical="bottom")	Bas (défaut)
Alignment(vertical="justify")	Justifié vertical
Alignment(vertical="distributed")	Distribué vertical
Alignment(wrap_text=True)	Renvoi à la ligne automatique
Alignment(shrink_to_fit=True)	Réduire pour adapter
Alignment(text_rotation=45)	Rotation du texte (0-180 degrés)
Alignment(text_rotation=255)	Vertical (de bas en haut)
Alignment(indent=1)	Retrait (multiple de 3 pts)
Alignment(indent=2)	Retrait double
Alignment(reading_order=1)	Gauche → droite
Alignment(reading_order=2)	Droite → gauche (RTL)
Alignment(relative_indent=0)	Retrait relatif

Application

```
cell.alignment = Alignment(horizontal="center", vertical="center", wrap_text=True)
```

— FORMAT NUMÉRIQUE —

cell.number_format = "General"	Général
cell.number_format = "0"	Entier
cell.number_format = "0.00"	2 décimales
cell.number_format = "#,##0"	Milliers sans décimales
cell.number_format = "#,##0.00"	Milliers avec 2 décimales
cell.number_format = "#,##0;(#,##0);-"	Avec négatifs entre parenthèses
cell.number_format = "0%"	Pourcentage entier
cell.number_format = "0.0%"	Pourcentage 1 décimale
cell.number_format = "0.00%"	Pourcentage 2 décimales
cell.number_format = "0.0x"	Multiple (valuation)
cell.number_format = "€#,##0.00"	Euros
cell.number_format = "\$#,##0.00"	Dollars
cell.number_format = "£#,##0.00"	Livres
cell.number_format = "yyyy-mm-dd"	Date ISO
cell.number_format = "dd/mm/yyyy"	Date française
cell.number_format = "mm/dd/yyyy"	Date américaine
cell.number_format = "dd/mm/yyyy hh:mm"	Date + heure
cell.number_format = "hh:mm:ss"	Heure
cell.number_format = "hh:mm"	Heure sans secondes
cell.number_format = "[hh]:mm:ss"	Durée (>24h)
cell.number_format = "@"	Texte (force format texte)
cell.number_format = "# ?/?"	Fraction
cell.number_format = "# ??/??"	Fraction double
cell.number_format = "0.00E+00"	Notation scientifique
cell.number_format = '"Oui";"Oui";"Non"'	Personnalisé positif/négatif/zéro

Couleurs dans le format

```
cell.number_format = '[Green]0.00;[Red]-0.00;[Black]-'  Couleur selon valeur
```

— PROTECTION CELLULE —

```

from openpyxl.styles import Protection
cell.protection = Protection(locked=True)    Cellule verrouillée
cell.protection = Protection(locked=False)   Cellule déverrouillée
cell.protection = Protection(hidden=True)     Formule masquée
cell.protection = Protection(locked=True, hidden=True)

```

— STYLES NOMMÉS

```
from openpyxl.styles import NamedStyle

style = NamedStyle(name="MonStyle")
style.font = Font(bold=True, color="1F4E79")
style.fill = PatternFill("solid", fgColor="D6E4F0")
style.border = Border(bottom=Side(style="thin"))
style.alignment = Alignment(horizontal="center")
style.number_format = "#,##0.00"
wb.add_named_style(style)           Enregistre dans le classeur
cell.style = "MonStyle"           Applique par nom
cell.style = style                 Applique par objet
```

```
# Styles intégrés Excel
cell.style = "Normal"
cell.style = "Bad"
cell.style = "Good"
cell.style = "Neutral"
cell.style = "Input"
cell.style = "Output"
cell.style = "Calculation"
cell.style = "Check Cell"
cell.style = "Explanatory Text"
cell.style = "Linked Cell"
cell.style = "Note"
cell.style = "Warning Text"
cell.style = "Title"
cell.style = "Heading 1"
cell.style = "Heading 2"
cell.style = "Heading 3"
cell.style = "Heading 4"
cell.style = "Total"
cell.style = "Accent1" .. "Accent6"
cell.style = "40% - Accent1" .. "40% - Accent6"
cell.style = "60% - Accent1" .. "60% - Accent6"
cell.style = "20% - Accent1" .. "20% - Accent6"
cell.style = "Comma"
cell.style = "Comma [0]"
cell.style = "Currency"
cell.style = "Currency [0]"
cell.style = "Percent"
```

EXEMPLES :

```
from openpyxl.styles import Font, PatternFill, Border, Side, Alignment

# Style en-tête
for cell in ws[1]:
    cell.font = Font(bold=True, color="FFFFFF", size=11)
    cell.fill = PatternFill("solid", fgColor="1F4E79")
    cell.alignment = Alignment(horizontal="center", vertical="center")
    cell.border = Border(bottom=Side(style="medium"))

# Lignes alternées
for i, row in enumerate(ws.iter_rows(min_row=2), start=2):
    couleur = "D9E1F2" if i % 2 == 0 else "FFFFFF"
    for cell in row:
        cell.fill = PatternFill("solid", fgColor=couleur)

# Total formaté
total_cell = ws.cell(row=ws.max_row+1, column=2)
total_cell.value = f"=SUM(B2:B{ws.max_row})"
total_cell.number_format = "#,##0.00"
total_cell.font = Font(bold=True)
total_cell.border = Border(top=Side(style="thin"), bottom=Side(style="double"))
```

[ws] Lignes & Colonnes – dimensions, groupes, filtres

DESCRIPTION : Manipule les dimensions, groupes et filtres de lignes/colonnes
POURQUOI : Mise en page, masquage, groupement et tri des données
CONTEXTE : Tableaux structurés, rapports hiérarchiques

LARGEUR DE COLONNE / HAUTEUR DE LIGNE :

```
ws.column_dimensions["A"].width = 20           Largeur colonne A (en caractères)
```

```

ws.column_dimensions["A"].width = 0          Largeur nulle = colonne masquée
ws.column_dimensions["A"].hidden = True      Masque la colonne
ws.column_dimensions["A"].auto_size = True   Auto-ajustement (pas garanti)
ws.column_dimensions["A"].best_fit = True    Meilleure taille
ws.column_dimensions["A"].customWidth = True Largeur personnalisée
ws.column_dimensions["A"].min = 1           Colonne de début pour groupes
ws.column_dimensions["A"].max = 1           Colonne de fin pour groupes
ws.column_dimensions["A"].outline_level = 1 Niveau de plan (0-7)
ws.column_dimensions["A"].collapsed = True   Groupe réduit

ws.row_dimensions[1].height = 30            Hauteur ligne 1 (en points)
ws.row_dimensions[1].hidden = True          Masque la ligne
ws.row_dimensions[1].customHeight = True    Hauteur personnalisée
ws.row_dimensions[1].outline_level = 1      Niveau de plan (0-7)
ws.row_dimensions[1].collapsed = True       Groupe réduit

AUTO-AJUSTEMENT LARGEUR (manuel) :
from openpyxl.utils import get_column_letter
for col in ws.columns:
    max_len = max(len(str(cell.value or "")) for cell in col)
    ws.column_dimensions[get_column_letter(col[0].column)].width = max_len + 2

INSERTION / SUPPRESSION :
ws.insert_rows(idx)                        Insère 1 ligne avant l'index idx
ws.insert_rows(idx, amount=3)              Insère 3 lignes
ws.delete_rows(idx)                        Supprime 1 ligne
ws.delete_rows(idx, amount=3)              Supprime 3 lignes
ws.insert_cols(idx)                        Insère 1 colonne avant idx
ws.insert_cols(idx, amount=2)              Insère 2 colonnes
ws.delete_cols(idx)                        Supprime 1 colonne
ws.delete_cols(idx, amount=2)              Supprime 2 colonnes
ws.move_range("A1:C3", rows=2, cols=1)     Déplace une plage
ws.move_range("A1:C3", rows=2, cols=1, translate=True) Avec translation formules

GROUPEMENT (PLAN) :
ws.column_dimensions["B"].outline_level = 1 Colonne B niveau 1
ws.column_dimensions["B"].collapsed = False Groupe déplié
ws.row_dimensions[5].outline_level = 1      Ligne 5 niveau 1
ws.row_dimensions[5].collapsed = False      Groupe déplié
ws.sheet_view.showOutlineSymbols            True = afficher symboles plan

FILTRES AUTO :
ws.auto_filter.ref = "A1:F1"               Active le filtre sur la ligne 1
ws.auto_filter.ref = ws.dimensions         Filtre sur toutes les données
ws.auto_filter.add_filter_column(0, ["Valeur1", "Valeur2"]) Filtre colonne 0
ws.auto_filter.add_sort_condition("A1:A100") Tri sur colonne A

TABLEAUX EXCEL (Tables) :
from openpyxl.worksheet.table import Table, TableStyleInfo
table = Table(displayName="Tableau1", ref="A1:D10")
style = TableStyleInfo(name="TableStyleMedium9", showFirstColumn=False,
                        showLastColumn=False, showRowStripes=True, showColumnStripes=False)
table.tableStyleInfo = style
ws.add_table(table)
ws.tables["Tableau1"]                      Accès par nom
for name, table in ws.tables.items():       Itération sur tous les tableaux

EXEMPLES :
# Ajustement automatique des colonnes
for column_cells in ws.columns:
    length = max(len(str(cell.value or "")) for cell in column_cells)
    ws.column_dimensions[column_cells[0].column_letter].width = length + 4

# Tableau formaté avec bandes
from openpyxl.worksheet.table import Table, TableStyleInfo
tab = Table(displayName="Données", ref=f"A1:E{ws.max_row}")
style = TableStyleInfo(name="TableStyleMedium2", showRowStripes=True)
tab.tableStyleInfo = style
ws.add_table(tab)

```

[ws] Formules – Formules Excel & noms définis

DESCRIPTION : Écrit des formules Excel et crée des plages nommées

POURQUOI : Calculs dynamiques, classeurs maintenables
CONTEXTE : Modèles financiers, rapports automatisés

FORMULES DE BASE :

ws["B10"] = "=SUM(B2:B9)"	Somme
ws["C10"] = "=AVERAGE(C2:C9)"	Moyenne
ws["D10"] = "=MAX(D2:D9)"	Maximum
ws["E10"] = "=MIN(E2:E9)"	Minimum
ws["F10"] = "=COUNT(F2:F9)"	Nombre
ws["G10"] = "=COUNTA(G2:G9)"	Nombre non vides
ws["H10"] = "=IF(A1>0,\"Oui\",\"Non\")"	Si conditionnel
ws["I10"] = "=VLOOKUP(A2,Feuil2!A:B,2,0)"	RechercheV
ws["J10"] = "=INDEX(B:B,MATCH(A1,A:A,0))"	Index/Equiv
ws["K10"] = "=CONCATENATE(A1,\" \",B1)"	Concaténer
ws["L10"] = "=A1&\" \"&B1"	Concaténer (opérateur &)
ws["M10"] = "=TEXT(A1,\"dd/mm/yyyy\")"	Formater date
ws["N10"] = "=TODAY()"	Date aujourd'hui
ws["O10"] = "=NOW()"	Date et heure actuelle
ws["P10"] = "=IFERROR(A1/B1,0)"	Gestion d'erreur
ws["Q10"] = "=ROUND(A1,2)"	Arrondir
ws["R10"] = "=SUMIF(A:A,\"X\",B:B)"	Somme conditionnelle
ws["S10"] = "=COUNTIF(A:A,\">0\")"	Comptage conditionnel
ws["T10"] = "=SUMPRODUCT(B2:B9,C2:C9)"	Somme des produits
ws["U10"] = "=LEN(A1)"	Longueur
ws["V10"] = "=LEFT(A1,3)"	3 caractères gauche
ws["W10"] = "=TRIM(A1)"	Supprimer espaces
ws["X10"] = "=UPPER(A1)"	Majuscules
ws["Y10"] = "=PROPER(A1)"	Majuscule initiale

RÉFÉRENCES ABSOLUES ET MIXTES :

ws["B2"] = "=A2*\$C\$1"	C1 absolu
ws["B2"] = "=A2*\$C1"	Colonne C absolue
ws["B2"] = "=A2*\$c1"	Ligne 1 absolue
ws["B2"] = "=Feuil2!A1"	Référence inter-feuilles
ws["B2"] = "='Nom avec espaces'!A1"	Feuille avec espaces

PLAGES NOMMÉES :

```
from openpyxl.workbook.defined_name import DefinedName
from openpyxl.utils import quote_sheetname

# Cellule unique
dn = DefinedName("TauxTVA", attr_text="Paramètres!$B$2")
wb.defined_names.add(dn)

# Plage
dn = DefinedName("Ventes", attr_text="Données!$B$2:$B$100")
wb.defined_names.add(dn)

# Niveau feuille
dn = DefinedName("Total", attr_text="Résumé!$D$10", localSheetId=0)
wb.defined_names.add(dn)

# Utilisation dans une formule
ws["E1"] = "=SUM(Ventes)*TauxTVA"

# Accès / suppression
wb.defined_names["TauxTVA"]          Accès par nom
del wb.defined_names["TauxTVA"]      Suppression
```

FORMULES ARRAY (Ctrl+Shift+Enter) :

```
from openpyxl.utils.dataframe import dataframe_to_rows
# Dans openpyxl, les formules array se saisissent normalement
# Excel les reconnaît à l'ouverture si syntaxe correcte
ws["A1"] = "=SUM(IF(B1:B10>0,B1:B10,0))"  Formule matricielle
```

EXEMPLES :

```
# Modèle financier simple
ws["B1"] = "2024"
ws["C1"] = "2025"
ws["A2"] = "Revenus"
ws["A3"] = "Charges"
ws["A4"] = "Résultat"
ws["B2"] = 100000
ws["B3"] = 70000
ws["B4"] = "=B2-B3"
```

```
ws["C2"] = "=B2*1.1"           Croissance 10%
ws["C3"] = "=B3*1.05"         Croissance 5%
ws["C4"] = "=C2-C3"
```

[ws] Mise en forme conditionnelle – ConditionalFormatting

DESCRIPTION : Applique des styles automatiquement selon les valeurs
 POURQUOI : Visualiser rapidement les données hors normes, tendances
 CONTEXTE : Tableaux de bord, indicateurs, heatmaps

RÈGLES BASIQUES :

```
from openpyxl.styles import PatternFill, Font, Color
from openpyxl.styles.differential import DifferentialStyle
from openpyxl.formatting.rule import (
    ColorScaleRule, DataBarRule, IconSetRule,
    CellIsRule, FormulaRule
)
```

RÈGLE DE CELLULE (CellIsRule) :

```
rouge = PatternFill(bgColor="FFC7CE")
rouge_font = Font(color="9C0006")
ds_rouge = DifferentialStyle(fill=rouge, font=rouge_font)
```

```
ws.conditional_formatting.add("A1:A100",
    CellIsRule(operator="lessThan", formula=["0"], dxf=ds_rouge))
# Opérateurs : "equal", "notEqual", "greaterThan", "greaterThanOrEqual",
#             "lessThan", "lessThanOrEqual", "between", "notBetween",
#             "containsText", "notContains", "beginsWith", "endsWith"
```

```
ws.conditional_formatting.add("B1:B100",
    CellIsRule(operator="between", formula=["0", "100"], dxf=ds_vert))
```

RÈGLE DE FORMULE (FormulaRule) :

```
vert = PatternFill(bgColor="C6EFCE")
ds_vert = DifferentialStyle(fill=vert)
ws.conditional_formatting.add("A1:D100",
    FormulaRule(formula=["$A1>AVERAGE($A$1:$A$100)"], dxf=ds_vert))
```

BARRE DE DONNÉES (DataBarRule) :

```
ws.conditional_formatting.add("C2:C100",
    DataBarRule(start_type="min", start_value=None,
        end_type="max", end_value=None,
        color="638EC6"))
```

NUANCES DE COULEUR (ColorScaleRule) :

```
ws.conditional_formatting.add("D2:D100",
    ColorScaleRule(start_type="min", start_color="F8696B", # Rouge
        mid_type="percentile", mid_value=50, mid_color="FFEB84", # Jaune
        end_type="max", end_color="63BE7B")) # Vert
```

2 couleurs

```
ws.conditional_formatting.add("E2:E100",
    ColorScaleRule(start_type="min", start_color="FFFFFF",
        end_type="max", end_color="4472C4"))
```

JEUX D'ICÔNES (IconSetRule) :

```
ws.conditional_formatting.add("F2:F100",
    IconSetRule(icon_style="3TrafficLights1",
        type="percent", values=[0, 33, 67]))
# Styles : "3Arrows", "3ArrowsGray", "3Flags", "3TrafficLights1",
#         "3TrafficLights2", "3Signs", "3Symbols", "3Symbols2",
#         "4Arrows", "4ArrowsGray", "4Rating", "4TrafficLights",
#         "5Arrows", "5ArrowsGray", "5Quarters", "5Rating"
```

DUPLIQUER / SUPPRIMER :

```
ws.conditional_formatting # Accès à toutes les règles
for cf in ws.conditional_formatting: # Itération
    print(cf.sqref, cf.rules)
```

EXEMPLES :

```
# Lignes entières selon condition colonne A
rouge_fill = PatternFill(bgColor="FFCCCC")
vert_fill = PatternFill(bgColor="CCFFCC")
```

```

ds_rouge = DifferentialStyle(fill=rouge_fill)
ds_vert = DifferentialStyle(fill=vert_fill)
ws.conditional_formatting.add("A2:F100",
    FormulaRule(formula=["$C2<0"], dxf=ds_rouge))
ws.conditional_formatting.add("A2:F100",
    FormulaRule(formula=["$C2>0"], dxf=ds_vert))

```

[ws] Graphiques – openpyxl.chart

DESCRIPTION : Crée des graphiques Excel intégrés dans la feuille
POURQUOI : Visualisation des données directement dans le fichier Excel
CONTEXTE : Rapports, tableaux de bord, présentations

TYPES DE GRAPHIQUES :

```

from openpyxl.chart import (
    BarChart, LineChart, PieChart, AreaChart,
    ScatterChart, BubbleChart, DoughnutChart,
    RadarChart, StockChart, SurfaceChart,
    BarChart3D, LineChart3D, PieChart3D, AreaChart3D
)

```

DONNÉES DE RÉFÉRENCE :

```

from openpyxl.chart import Reference, Series

Reference(ws, min_col=2, min_row=1, max_col=4, max_row=10)  Plage
Reference(ws, min_col=1, min_row=2, max_row=10)             Colonne
Reference(ws, min_col=1, min_row=1)                          Cellule unique

```

GRAPHIQUE EN BARRES (BarChart) :

```

chart = BarChart()
chart.type = "col"          Barres verticales (colonnes)
chart.type = "bar"          Barres horizontales
chart.grouping = "clustered" Groupé
chart.grouping = "stacked"   Empilé
chart.grouping = "percentStacked" Empilé 100%
chart.overlap = 100          Chevauchement (stacked=100)
chart.title = "Mon graphique"
chart.y_axis.title = "Valeurs"
chart.x_axis.title = "Catégories"
chart.style = 10              Style prédéfini (1-48)
chart.width = 15              Largeur en cm
chart.height = 10             Hauteur en cm

```

```

data = Reference(ws, min_col=2, max_col=4, min_row=1, max_row=10)
cats = Reference(ws, min_col=1, min_row=2, max_row=10)
chart.add_data(data, titles_from_data=True)
chart.set_categories(cats)
ws.add_chart(chart, "F2")      Position d'ancrage

```

GRAPHIQUE EN LIGNES (LineChart) :

```

chart = LineChart()
chart.style = 10
chart.title = "Évolution"
chart.y_axis.crossAx = 500
chart.x_axis = DateAxis()      Axe de dates
chart.x_axis.numFmtId = 14      Format date
chart.x_axis.scaling.orientation = "minMax"

```

```

data = Reference(ws, min_col=2, max_col=4, min_row=1, max_row=10)
chart.add_data(data, titles_from_data=True)
# Personnaliser les séries
for i, series in enumerate(chart.series):
    series.smooth = True        Courbe lissée
    series.marker.symbol = "circle" Marqueur
    series.marker.size = 5

```

GRAPHIQUE CIRCULAIRE (PieChart) :

```

chart = PieChart()
chart.title = "Répartition"
labels = Reference(ws, min_col=1, min_row=2, max_row=6)
data = Reference(ws, min_col=2, min_row=1, max_row=6)
chart.add_data(data, titles_from_data=True)
chart.set_categories(labels)

```

chart.dataLabels = DataLabel()	
chart.dataLabels.showPercent = True	Afficher pourcentages
chart.dataLabels.showCatName = True	Afficher catégories

GRAPHIQUE EN AIRES (AreaChart) :

```
chart = AreaChart()
chart.grouping = "stacked"
```

GRAPHIQUE EN NUAGE (ScatterChart) :

```
chart = ScatterChart()
chart.style = 13
chart.title = "Corrélation"
chart.x_axis.title = "X"
chart.y_axis.title = "Y"
xvalues = Reference(ws, min_col=1, min_row=2, max_row=10)
yvalues = Reference(ws, min_col=2, min_row=2, max_row=10)
series = Series(yvalues, xvalues, title="Série 1")
series.marker.symbol = "circle"
series.graphicalProperties.line.noFill = True Sans ligne
chart.series.append(series)
```

PROPRIÉTÉS COMMUNES :

chart.title	Titre du graphique
chart.width	Largeur (cm, défaut 15)
chart.height	Hauteur (cm, défaut 7.5)
chart.style	Style 1-48
chart.legend.position	"b", "l", "r", "t", "tr"
chart.legend = None	Supprimer la légende
chart.plot_area	Zone de tracé
chart.y_axis.title	Titre axe Y
chart.x_axis.title	Titre axe X
chart.y_axis.numFmt	Format de l'axe Y
chart.y_axis.scaling.min	Valeur min de l'axe
chart.y_axis.scaling.max	Valeur max de l'axe
chart.y_axis.delete = False	Afficher/masquer l'axe
chart.y_axis.majorGridlines	Lignes de grille principales
chart.y_axis.minorGridlines	Lignes de grille secondaires
chart.series[0].title.value	Nom d'une série
chart.series[0].graphicalProperties.solidFill	Couleur de remplissage
chart.series[0].graphicalProperties.line.solidFill	Couleur de ligne

EXEMPLES :

```
from openpyxl.chart import BarChart, Reference
chart = BarChart()
chart.type = "col"
chart.title = "Ventes par mois"
chart.y_axis.title = "€"
chart.x_axis.title = "Mois"
data = Reference(ws, min_col=2, max_col=3, min_row=1, max_row=13)
cats = Reference(ws, min_col=1, min_row=2, max_row=13)
chart.add_data(data, titles_from_data=True)
chart.set_categories(cats)
chart.style = 10
chart.width = 20
chart.height = 12
ws.add_chart(chart, "E2")
```

[ws] Images – openpyxl.drawing.image

DESCRIPTION : Insère des images dans une feuille Excel
POURQUOI : Logos, photos, illustrations dans les rapports
CONTEXTE : En-têtes de rapport, fiches produit

INSERTION :

```
from openpyxl.drawing.image import Image

img = Image("logo.png") Charge une image
img = Image("photo.jpg") JPEG
img.width = 150 Largeur en pixels
img.height = 100 Hauteur en pixels
img.anchor = "B2" Position d'ancrage
ws.add_image(img, "B2") Insère à la position B2
ws.add_image(img) Insère (ancre via img.anchor)
```

```
# Redimensionnement proportionnel
ratio = img.height / img.width
img.width = 200
img.height = int(200 * ratio)
```

EXEMPLES :

```
logo = Image("assets/logo.png")
logo.width, logo.height = 120, 40
ws.add_image(logo, "A1")
```

[ws] Validation des données – DataValidation

DESCRIPTION : Restreint les valeurs saisies dans des cellules
 POURQUOI : Prévenir les erreurs, créer des listes déroulantes
 CONTEXTE : Formulaire, saisies guidées, menus déroulants

CRÉATION :

```
from openpyxl.worksheet.datavalidation import DataValidation

dv = DataValidation(type="list", formula1="Option1,Option2,Option3'")
dv = DataValidation(type="list", formula1="Feuil2!$A$1:$A$10")      Liste depuis plage
dv = DataValidation(type="whole", operator="between",
                    formula1="1", formula2="100")                  Entier 1-100
dv = DataValidation(type="decimal", operator="greaterThan",
                    formula1="0")                                  Décimal > 0
dv = DataValidation(type="date", operator="greaterThanOrEqual",
                    formula1="2020-01-01")                        Date >= 2020
dv = DataValidation(type="time")                                  Heure
dv = DataValidation(type="textLength", operator="lessThanOrEqual",
                    formula1="50")                                Texte ≤ 50 car
dv = DataValidation(type="custom",
                    formula1="=ISNUMBER(A1)")                     Formule custom
```

OPTIONS :

```
dv.showDropDown = False      True = masquer flèche liste déroulante
dv.showInputMessage = True   Afficher message de saisie
dv.promptTitle = "Aide"      Titre du message
dv.prompt = "Entrez une valeur entre 1 et 100" Texte du message
dv.showErrorAlert = True     Afficher alerte erreur
dv.errorTitle = "Valeur incorrecte" Titre de l'alerte
dv.error = "Veuillez entrer une valeur valide" Message d'erreur
dv.errorStyle = "stop"       "stop", "warning", "information"
dv.allow_blank = True        Autoriser valeur vide
```

APPLICATION :

```
ws.add_data_validation(dv)      Ajoute la validation à la feuille
dv.add("A1")                    Ajoute une cellule
dv.add("A1:A100")               Ajoute une plage
dv.sqref                        Plage couverte
```

EXEMPLES :

```
# Liste déroulante simple
dv = DataValidation(type="list", formula1="Oui,Non,En attente'",
                    showDropDown=False)
dv.prompt = "Choisissez un statut"
dv.showInputMessage = True
ws.add_data_validation(dv)
dv.add(f"D2:D{ws.max_row}")

# Entier entre 0 et 100
dv2 = DataValidation(type="whole", operator="between",
                    formula1="0", formula2="100",
                    showErrorAlert=True, errorStyle="stop",
                    errorTitle="Erreur", error="Valeur entre 0 et 100 requis")
ws.add_data_validation(dv2)
dv2.add("E2:E100")
```

[op] Utilitaires – openpyxl.utils & openpyxl.utils.dataframe

DESCRIPTION : Fonctions utilitaires, conversion colonnes, intégration pandas

POURQUOI : Faciliter la conversion d'indices, import/export de DataFrames
CONTEXTE : Pipelines de données, rapports automatisés

UTILS DE COLONNES / CELLULES :

```
from openpyxl.utils import (
    get_column_letter,
    column_index_from_string,
    coordinate_to_tuple,
    absolute_coordinate,
    quote_sheetname,
    rows_from_range,
    cols_from_range,
    range_boundaries,
    range_to_tuple,
)

get_column_letter(1)           "A"
get_column_letter(702)        "ZZ"
column_index_from_string("A")  1
column_index_from_string("ZZ") 702
coordinate_to_tuple("C5")      (5, 3) # (row, col)
absolute_coordinate("A1:B2")   "$A$1:$B$2"
quote_sheetname("Feuille 1")   "'Feuille 1'"
rows_from_range("A1:C3")       Générateur de tuples de coordonnées
cols_from_range("A1:C3")       Générateur de colonnes
range_boundaries("A1:C3")      (1, 1, 3, 3) # (min_c,min_r,max_c,max_r)
```

INTÉGRATION PANDAS :

```
from openpyxl.utils.dataframe import dataframe_to_rows

# DataFrame → Worksheet
import pandas as pd
df = pd.DataFrame({"A": [1, 2, 3], "B": [4, 5, 6]})
for r in dataframe_to_rows(df, index=False, header=True):
    ws.append(r)

# Avec index
for r in dataframe_to_rows(df, index=True, header=True):
    ws.append(r)

# Worksheet → DataFrame
data = ws.values
cols = next(data) # Première ligne = en-têtes
df = pd.DataFrame(data, columns=cols)

# Chargement direct avec pandas (plus simple)
df = pd.read_excel("fichier.xlsx", sheet_name="Feuille1")
df.to_excel("sortie.xlsx", index=False, sheet_name="Données")
```

EXEMPLES :

```
# Écriture DataFrame avec styles
from openpyxl.utils.dataframe import dataframe_to_rows
df = pd.DataFrame(données)
wb = Workbook()
ws = wb.active

# En-tête avec style
en_tetes = list(df.columns)
ws.append(en_tetes)
for cell in ws[1]:
    cell.font = Font(bold=True, color="FFFFFF")
    cell.fill = PatternFill("solid", fgColor="1F4E79")

# Données
for row in dataframe_to_rows(df, index=False, header=False):
    ws.append(row)

wb.save("rapport.xlsx")
```

```

from openpyxl import Workbook
from openpyxl.styles import Font, PatternFill, Border, Side, Alignment
from openpyxl.utils import get_column_letter

wb = Workbook()
ws = wb.active
ws.title = "Rapport"

# En-têtes
colonnes = ["Produit", "Q1", "Q2", "Q3", "Q4", "Total"]
ws.append(colonnes)
for cell in ws[1]:
    cell.font = Font(bold=True, color="FFFFFF", size=11)
    cell.fill = PatternFill("solid", fgColor="1F4E79")
    cell.alignment = Alignment(horizontal="center")
    cell.border = Border(bottom=Side(style="medium"))

# Données
produits = [
    ["Produit A", 100, 120, 110, 130],
    ["Produit B", 200, 180, 220, 190]
]
for i, ligne in enumerate(produits, start=2):
    ws.append(ligne)
    ws.cell(row=i, column=6).value = f"=SUM(B{i}:E{i})"
    # Bandes alternées
    couleur = "D9E1F2" if i % 2 == 0 else "FFFFFF"
    for cell in ws[i]:
        cell.fill = PatternFill("solid", fgColor=couleur)
        cell.alignment = Alignment(horizontal="center")

# Ligne total
total_row = ws.max_row + 1
ws.cell(row=total_row, column=1).value = "TOTAL"
for col in range(2, 7):
    c = ws.cell(row=total_row, column=col)
    c.value = f"=SUM({get_column_letter(col)}2:{get_column_letter(col)}{total_row-1})"
    c.font = Font(bold=True)
    c.border = Border(top=Side(style="thin"), bottom=Side(style="double"))
    c.number_format = "#,##0"

# Ajustement colonnes
for col in ws.columns:
    length = max(len(str(cell.value or "")) for cell in col) + 4
    ws.column_dimensions[col[0].column_letter].width = length

# Gel de la première ligne
ws.freeze_panes = "A2"

wb.save("rapport_complet.xlsx")

```

— CLASSEUR MULTI-FEUILLES

```

wb = Workbook()
wb.remove(wb.active)                                Supprime la feuille par défaut

for nom, données in données_par_feuille.items():
    ws = wb.create_sheet(nom)
    ws.tab_color = couleurs[nom]
    for ligne in données:
        ws.append(ligne)

# Feuille résumé
résumé = wb.create_sheet("Résumé", 0)
for i, nom in enumerate(wb.sheetnames[1:], start=2):
    résumé.cell(row=i, column=1).value = nom
    résumé.cell(row=i, column=2).value = f"='{nom}!'B1"

```

— COPIER LE FORMAT D'UNE CELLULE

```

import copy
def copier_style(source, dest):
    dest.font = copy.copy(source.font)
    dest.fill = copy.copy(source.fill)
    dest.border = copy.copy(source.border)
    dest.alignment = copy.copy(source.alignment)
    dest.number_format = source.number_format
    dest.protection = copy.copy(source.protection)

```

— EXPORT RAPIDE DEPUIS LISTE DE DICTS —

```
def liste_dicts_vers_excel(données, chemin):
    wb = Workbook()
    ws = wb.active
    if not données:
        wb.save(chemin); return
    # En-têtes
    colonnes = list(données[0].keys())
    ws.append(colonnes)
    for cell in ws[1]:
        cell.font = Font(bold=True)
    # Données
    for ligne in données:
        ws.append([ligne.get(col) for col in colonnes])
    # Auto-largeur
    for col in ws.columns:
        max_len = max(len(str(c.value or "")) for c in col)
        ws.column_dimensions[col[0].column_letter].width = max_len + 2
    wb.save(chemin)
```

— PROTECTION SÉLECTIVE —

```
# 1. Déverrouiller tout
unlock = Protection(locked=False)
for row in ws.iter_rows():
    for cell in row:
        cell.protection = unlock

# 2. Verrouiller les cellules à protéger
for cell in ws["A1:F1"][0]:
    cell.protection = Protection(locked=True)

# 3. Activer la protection de la feuille
ws.protection.enable()
ws.protection.set_password("secret")
```

RÉCAPITULATIF DES IMPORTS OPENPYXL

IMPORT	USAGE
from openpyxl import Workbook, load_workbook	Core : créer / charger classeur
from openpyxl.styles import Font	Police (gras, italique, couleur...)
from openpyxl.styles import PatternFill	Remplissage de fond
from openpyxl.styles import GradientFill	Remplissage dégradé
from openpyxl.styles import Border, Side	Bordures de cellules
from openpyxl.styles import Alignment	Alignement et retour à la ligne
from openpyxl.styles import Protection	Verrouillage de cellules
from openpyxl.styles import NamedStyle	Styles nommés réutilisables
from openpyxl.styles import Color	Objet couleur
from openpyxl.styles.differential import DifferentialStyle	MFC (diff styles)
from openpyxl.formatting.rule import CellIsRule	Règle MFC valeur cellule
from openpyxl.formatting.rule import FormulaRule	Règle MFC formule
from openpyxl.formatting.rule import ColorScaleRule	Nuances de couleur
from openpyxl.formatting.rule import DataBarRule	Barres de données
from openpyxl.formatting.rule import IconSetRule	Jeux d'icônes
from openpyxl.chart import BarChart, LineChart, PieChart	Graphiques
from openpyxl.chart import Reference, Series	Données de graphiques
from openpyxl.chart.axis import DateAxis	Axe de dates
from openpyxl.chart.label import DataLabel	Étiquettes de données
from openpyxl.drawing.image import Image	Images
from openpyxl.worksheet.datavalidation import DataValidation	Validation données
from openpyxl.worksheet.table import Table, TableStyleInfo	Tableaux Excel
from openpyxl.workbook.defined_name import DefinedName	Plages nommées
from openpyxl.comments import Comment	Commentaires
from openpyxl.utils import get_column_letter	Lettre de colonne
from openpyxl.utils import column_index_from_string	Index de colonne
from openpyxl.utils import coordinate_to_tuple	Coord → tuple
from openpyxl.utils import range_boundaries	Limites d'une plage
from openpyxl.utils import quote_sheetname	Quoter nom de feuille
from openpyxl.utils.dataframe import dataframe_to_rows	DataFrame → lignes
from openpyxl.worksheet.page import Break	Sauts de page

— MODES D'OUVERTURE —

MODE	USAGE	LIMITATION
------	-------	------------

load_workbook()	Lecture + écriture	Charge tout en RAM
load_workbook(read_only=True)	Lecture rapide	Pas de modification
load_workbook(data_only=True)	Valeurs calculées	Formules non conservées
Workbook(write_only=True)	Écriture seule	Pas de lecture, streaming
Workbook()	Création standard	Usage général

— TYPES DE DONNÉES CELL.DATA_TYPE —		
CODE	TYPE	EXEMPLE
"n"	Numérique	42, 3.14, -100
"s"	Chaîne	"Texte", "Bonjour"
"b"	Booléen	True, False
"d"	Date	datetime(2024, 1, 1)
"f"	Formule	"=SUM(A1:A10)"
"e"	Erreur	"#REF!", "#DIV/0!"
"inlineStr"	Chaîne inline (mode read_only)	
None	Vide	None

— PAPIERS D'IMPRESSION (page_setup.paperSize) —	
VALEUR	FORMAT
1	Letter (8.5 x 11 pouces)
2	Letter small
5	Legal
8	A3
9	A4
10	A4 small
11	A5
12	B4
13	B5
17	A4 extra

— CHECKLIST ANTI-ERREURS —	
VÉRIFICATION	RAISON
Indices 1-based	row=1, column=1 → A1 (pas 0)
convert() après data_only=True	Formules remplacées si sauvegardé
copy.copy() pour copier un style	Les styles sont des objets mutables
del px avant toute opération	PixelArray (N/A openpyxl, mais important)
recalc.py après ajout de formules	openpyxl n'évalue pas les formules
quote_sheetname() pour noms avec espaces	'Ma Feuille'!A1 pas Ma Feuille!A1
iter_rows values_only=True pour perf	Évite de créer des objets Cell inutiles
write_only pour gros fichiers	Évite les erreurs mémoire (>100k lignes)
wb.close() en production	Libère les handles de fichiers

=====